

Forecasting Realized Volatility with Graph Spillovers and Implied Volatility: A GNNHAR-IV Extension

Manuscript for Review

June 4, 2026

Abstract

This paper extends the graph neural network heterogeneous autoregressive framework of [Zhang et al. \(2025\)](#) by adding implied volatility (IV) as a forward-looking information channel. The original GNNHAR framework studies whether realized-volatility forecasts improve when the HAR model is augmented with graph-based spillover effects, multi-hop graph aggregation, nonlinear graph neural networks, and alternative estimation criteria such as MSE and QLIKE. We preserve that evaluation logic and construct HAR-IV, GHAR-IV, and GNNHAR-IV models for Dow 30 realized volatility forecasting. To distinguish genuine IV information from mere parameter expansion, we compare real-IV models with fake-IV controls that preserve the scale and dimension of IV features while breaking their time alignment. The IV source is AlphaQuery’s stock-level implied-volatility data product, using the 30-day implied-volatility mean as the forward-looking option-market signal. The reproducible run covers 30 Dow constituents from 2021-02-26 to 2026-01-26, uses a chronological train-validation-test split, builds a GLASSO return network, and evaluates all forecasts using out-of-sample MSE, QLIKE, relative loss ratios, model confidence sets, Diebold-Mariano tests, and calm-versus-volatile regime splits. The best model by test QLIKE is GHAR-IV, with QLIKE 0.0007849, while the best model by test MSE is the validation-tuned GNNHAR3L-IV, with MSE 0.9591. The IV decomposition shows that real IV carries economically meaningful predictive information for linear HAR and GHAR specifications, and validation tuning makes the nonlinear GNNHAR-IV extension competitive with the strongest linear benchmark. The results position IV as a useful extension to Zhang et al.’s graph-spillover volatility framework while also showing that neural graph models require careful validation-based tuning before they can be interpreted in small-sample empirical finance settings.

Keywords: realized volatility; implied volatility; graph neural networks; HAR; GHAR; GNNHAR; QLIKE; model confidence set; volatility spillovers.

1 Introduction

Forecasting equity volatility is central to risk management, derivatives valuation, portfolio allocation, and stress testing. The heterogeneous autoregressive model of [Corsi \(2009\)](#) remains a strong benchmark because it expresses persistent realized volatility dynamics through daily, weekly, and monthly lag components. A limitation of the classical HAR specification is that it is largely asset-by-asset: it captures own-volatility persistence but does not explicitly represent volatility spillovers across assets. [Zhang et al. \(2025\)](#) address this limitation by embedding

HAR features in a graph-based framework. Their study starts from graph HAR (GHAR), where neighbors are defined through a financial network, and develops GNNHAR models that use graph neural networks to capture nonlinear and multi-hop spillover effects. They evaluate forecasts with MSE and QLIKE, model confidence sets, pairwise forecast loss tests, and regime-specific comparisons.

This paper asks a complementary question. If graph spillovers improve the way past realized volatility is organized across assets, can the same framework benefit from a forward-looking market signal? We study this question by adding implied volatility (IV) to the GNNHAR framework. IV, extracted from option prices, reflects the market’s forward-looking volatility assessment and is therefore conceptually different from realized volatility, which is backward looking. The resulting extension, denoted GNNHAR-IV, augments the original realized-volatility features with IV lag features at the same daily, weekly, and monthly HAR horizons. This design keeps the structure of Zhang et al.’s framework while adding a new economic information channel.

The key empirical concern is that IV models contain more regressors and neural-network input features. Apparent gains could therefore arise from parameter expansion rather than genuine IV information. To address this, we introduce fake-IV controls. Fake IV is obtained by permuting the IV feature tensor over dates, preserving the dimension and scale of the IV channel while destroying its timing relation with future realized volatility. Comparing no-IV, real-IV, and fake-IV models allows us to decompose the improvement into total IV gain, genuine information gain, and parameter expansion gain.

Our contribution is threefold. First, we provide a reproducible Colab and private GitHub pipeline that implements HAR, GHAR, GNNHAR1L, GNNHAR2L, GNNHAR3L, and their IV counterparts on a Dow 30 panel. Second, we evaluate the IV extension using the same style of evidence emphasized by Zhang et al. (2025): MSE, QLIKE, relative loss ratios, model confidence sets (MCS), Diebold-Mariano (DM) tests, regime splits, and forecast distribution plots. Third, we show that IV improves linear HAR/GHAR specifications in a way that survives the fake-IV benchmark, and that validation-tuned GNNHAR-IV can outperform linear graph benchmarks by MSE while remaining close to GHAR-IV by QLIKE.

The main finding is that GHAR-IV is the best-performing model by QLIKE in the tuned run, achieving a test QLIKE of 0.0007849 and a QLIKE ratio of 0.9549 relative to HAR. The validation-tuned GNNHAR3L-IV is the best-performing model by MSE, achieving test MSE 0.9591, below GHAR-IV’s 1.0272 and GHAR’s 1.0848. HAR-IV also improves on HAR, but GHAR-IV is better than HAR-IV, indicating that IV and graph spillovers are complementary. The fake-IV experiment strengthens the interpretation: for HAR and GHAR, real-IV gains are much larger than fake-IV gains. Neural graph models require careful validation tuning: without tuning they are fragile, while with validation-selected hyperparameters the IV-augmented GNN variants become competitive. We interpret this as evidence for careful tuning rather than as evidence against graph spillovers or IV.

2 Relation to Zhang et al.’s GNNHAR Framework

Zhang et al. (2025) propose a customized GNN framework for forecasting multivariate realized

volatilities with spillover effects. Their paper emphasizes three empirical questions: whether multi-hop neighbors add value, whether nonlinear spillovers improve forecasts, and whether QLIKE training can outperform MSE training because of heteroskedastic volatility errors. Their baseline comparison includes HAR, GHAR, and GNNHAR models with one, two, and three graph layers. Forecasts are evaluated out of sample by both MSE and QLIKE, and statistical evidence is summarized through loss ratios, MCS tests, DM-type comparisons, and market-regime analysis.

The present paper follows this architecture but modifies the information set. Let $v_{i,t}$ denote realized volatility of asset i on day t , and let $x_{i,t}$ denote its implied volatility. The original GNNHAR framework uses lagged $v_{i,t}$ and graph-aggregated lagged realized volatility. We add lagged $x_{i,t}$ and graph-aggregated lagged IV. The economic motivation is straightforward: if option prices contain forward-looking information about expected volatility, then IV should help predict future RV even after conditioning on historical realized volatility and spillover effects. The econometric challenge is to distinguish that information from a larger feature set.

3 Model Specification

3.1 HAR and HAR-IV Features

For each asset i , the standard HAR feature vector contains daily, weekly, and monthly realized-volatility components:

$$H_{i,t-1}^{RV} = \left(v_{i,t-1}, \frac{1}{5} \sum_{\ell=1}^5 v_{i,t-\ell}, \frac{1}{22} \sum_{\ell=1}^{22} v_{i,t-\ell} \right)'.$$

The IV extension mirrors this construction:

$$H_{i,t-1}^{IV} = \left(x_{i,t-1}, \frac{1}{5} \sum_{\ell=1}^5 x_{i,t-\ell}, \frac{1}{22} \sum_{\ell=1}^{22} x_{i,t-\ell} \right)'.$$

The HAR and HAR-IV models are

$$\hat{v}_{i,t}^{HAR} = \alpha_i + \beta_i' H_{i,t-1}^{RV},$$

and

$$\hat{v}_{i,t}^{HAR-IV} = \alpha_i + \beta_i' H_{i,t-1}^{RV} + \delta_i' H_{i,t-1}^{IV}.$$

In the implementation, the linear models are estimated as pooled ridge regressions with standardized features, which stabilizes coefficient estimates in the panel setting.

3.2 GLASSO Graph Construction

Following the graph-spillover logic of [Zhang et al. \(2025\)](#), the network is estimated from training-window stock returns. Let R_t be the vector of daily returns, S the sample covariance matrix,

and Θ the precision matrix. The graphical lasso solves

$$\hat{\Theta} = \arg \min_{\Theta \succ 0} \{ \text{tr}(S\Theta) - \log \det(\Theta) + \lambda \|\Theta\|_1 \}.$$

An undirected adjacency matrix A is constructed by setting $A_{ij} = 1$ when $\hat{\Theta}_{ij} \neq 0$ and $i \neq j$. The normalized graph operator is

$$W = D^{-1/2} A D^{-1/2},$$

where D is the degree matrix. The empirical run selected GLASSO regularization $\lambda = 0.159968$ and produced edge density 0.442222.

3.3 GHAR-IV

GHAR augments HAR with graph-aggregated realized-volatility features:

$$\hat{v}_{i,t}^{GHAR} = \alpha_i + \beta'_i H_{i,t-1}^{RV} + \gamma'_i (W H_{t-1}^{RV})_i.$$

The IV extension includes both own IV and graph-aggregated IV:

$$\hat{v}_{i,t}^{GHAR-IV} = \alpha_i + \beta'_i H_{i,t-1}^{RV} + \gamma'_i (W H_{t-1}^{RV})_i + \delta'_i H_{i,t-1}^{IV} + \eta'_i (W H_{t-1}^{IV})_i.$$

This specification is the most direct extension of Zhang et al.'s GHAR baseline because it retains graph spillovers while adding a forward-looking information channel.

3.4 GNNHAR-IV

The GNNHAR model replaces the linear graph aggregation in GHAR with nonlinear graph convolutional layers. Let $Z_t^{(0)}$ denote the input feature tensor. In the no-IV model, $Z_t^{(0)} = H_{t-1}^{RV}$; in the IV model, $Z_t^{(0)} = [H_{t-1}^{RV}, H_{t-1}^{IV}]$. A graph layer is represented as

$$Z_t^{(\ell+1)} = \sigma \left(W Z_t^{(\ell)} B^{(\ell)} + b^{(\ell)} \right),$$

where $B^{(\ell)}$ is a trainable weight matrix and $\sigma(\cdot)$ is a ReLU nonlinearity. The output layer maps node embeddings to positive volatility forecasts using a softplus link. One, two, and three graph layers correspond to GNNHAR1L, GNNHAR2L, and GNNHAR3L, respectively. This multi-layer design parallels Zhang et al.'s multi-hop setup. The empirical implementation therefore includes both no-IV neural graph models and their IV-augmented counterparts:

$$\text{GNNHAR}k\text{L-IV} : \quad Z_t^{(0)} = [H_{t-1}^{RV}, H_{t-1}^{IV}], \quad k \in \{1, 2, 3\}.$$

These models are not linear GHAR-IV regressions. They are nonlinear graph neural networks whose first-layer inputs contain both realized-volatility HAR features and AlphaQuery IV HAR features, and whose graph propagation matrix is the same GLASSO network used in GHAR.

3.5 Forecast Losses and Estimation Criteria

Forecast performance is evaluated using MSE and QLIKE:

$$L_{MSE}(v, \hat{v}) = (v - \hat{v})^2,$$

$$L_{QLIKE}(v, \hat{v}) = \frac{v}{\hat{v}} - \log\left(\frac{v}{\hat{v}}\right) - 1,$$

with forecasts clipped away from zero. The distinction between estimation criterion and forecast loss is important. Linear models are estimated by MSE, while the neural models include both MSE-trained and QLIKE-trained variants. All models are evaluated out of sample using both MSE and QLIKE.

3.6 Fake-IV Decomposition

Let L_0 , L_{IV} , and L_{fake} denote the test loss of a no-IV model, its real-IV extension, and its fake-IV extension. We report

$$\Delta_{total} = L_0 - L_{IV}, \quad \Delta_{info} = L_{fake} - L_{IV}, \quad \Delta_{param} = L_0 - L_{fake}.$$

Δ_{total} is the total IV improvement, Δ_{info} is the genuine information gain, and Δ_{param} is the parameter expansion gain. A credible IV result should have $\Delta_{info} > 0$, not merely $\Delta_{total} > 0$.

4 Data and Experimental Design

The empirical analysis uses Dow 30 constituents with realized volatility, implied volatility, and daily returns. The implied-volatility input is sourced from AlphaQuery’s volatility and option statistics pages ([AlphaQuery, 2026](#)). Specifically, the IV channel uses the 30-day “Implied Volatility (Mean)” field, which AlphaQuery reports as the average of put and call implied volatilities for options with the relevant expiration date. AlphaQuery also reports the corresponding 30-day call IV, put IV, put-call IV ratio, skew, and historical-volatility measures; this study uses the mean IV series as the primary forward-looking option-market signal. After HAR lag construction, the sample contains 1234 trading dates from 2021-02-26 to 2026-01-26. The chronological split uses 863 training dates, 185 validation dates, and 186 test dates. All reported tables are produced by the repository pipeline on branch 2026-06-01. The tuned run metadata confirms 250 base neural training epochs, 60 MCS bootstrap replications, seed 20260602, `fast=false`, `tune_gnn=true`, and `skip_qlike_training=true`. The Colab notebook in `notebooks/gnnhar_iv_colab_full_run.ipynb` has been updated to run the same validation-tuned pipeline and to push generated artifacts back to the private GitHub repository. The exported tables contain HAR, GHAR, GNNHAR1L, GNNHAR2L, GNNHAR3L, GNNHAR1L-IV, GNNHAR2L-IV, GNNHAR3L-IV, fake-IV controls, random-graph controls, and the GNN hyperparameter-selection table.

Table 1: Data and full-run configuration

Item	Value
Universe	Dow 30 constituents
Number of assets	30
Sample after lags	2021-02-26 to 2026-01-26
IV data source	AlphaQuery 30-day IV Mean
Neural model family	GNNHAR1L/2L/3L with and without IV
Training dates	863
Validation dates	185
Test dates	186
Graph construction	GLASSO on training returns
GLASSO alpha	0.159968
Graph edge density	0.442222
Neural base epochs	250
GNN hyperparameter selection	Validation split
MCS bootstrap replications	60

Table 2: Out-of-sample forecast performance

Model	Test MSE	Test QLIKE	MSE ratio/HAR	QLIKE ratio/HAR
GHAR-IV	1.0272	0.0007849	0.9114	0.9549
HAR-IV	1.0464	0.0007928	0.9285	0.9645
GNNHAR3L-IV	0.9591	0.0007964	0.8510	0.9689
GHAR	1.0848	0.0008111	0.9626	0.9868
GNNHAR2L-IV	1.0180	0.0008120	0.9032	0.9878
GHAR-fakeIV	1.0897	0.0008136	0.9669	0.9898
GHAR-random	1.0901	0.0008142	0.9672	0.9905
HAR-fakeIV	1.1217	0.0008207	0.9953	0.9985
HAR	1.1270	0.0008220	1.0000	1.0000

Notes: The table reports test-period averages across dates and assets. The suffix “fakeIV” denotes a date-permuted IV control. GHAR-random uses a random graph with comparable density. GNNHAR rows use hyperparameters selected on the validation split, not on the test split.

5 Empirical Results

5.1 Main Forecast Comparison

Table 2 reports the main out-of-sample comparison. GHAR-IV is the best model by QLIKE, while validation-tuned GNNHAR3L-IV is the best model by MSE. The no-IV graph model GHAR improves over HAR, but adding IV produces a larger improvement. Relative to HAR, GHAR-IV reduces MSE by about 8.86% and QLIKE by about 4.51%. GNNHAR3L-IV reduces MSE by about 14.90% relative to HAR and by about 6.63% relative to HAR-IV, while its QLIKE is nearly tied with GHAR-IV. These results support the view that IV and graph spillovers are complementary, and that nonlinear graph layers can add value after validation tuning.

Table 3: GNNHAR and GNNHAR-IV forecast performance

Model	Estimation	Test MSE	Test QLIKE	QLIKE ratio/HAR
GNNHAR3L-IV	MSE	0.9591	0.0007964	0.9689
GNNHAR2L-IV	MSE	1.0180	0.0008120	0.9878
GNNHAR1L-IV	MSE	1.0369	0.0008348	1.0156
GNNHAR1L	MSE	1.1485	0.0008476	1.0312
GNNHAR2L	MSE	1.1326	0.0008616	1.0483
GNNHAR1L-IV-fakeIV	MSE	1.1786	0.0008702	1.0587
GNNHAR3L	MSE	1.1905	0.0008962	1.0903

Table 4: Model confidence set at the 5 percent level

Model	Included	MCS p-value
GHAR-IV	Yes	1.000
HAR-IV	Yes	0.833
GNNHAR3L-IV	Yes	0.833
HAR	Yes	0.417
GNNHAR2L-IV	Yes	0.400
GHAR	Yes	0.383
HAR-fakeIV	Yes	0.383
GHAR-random	Yes	0.317
GHAR-fakeIV	Yes	0.317
GNNHAR1L-IV	Yes	0.317
GNNHAR1L-IV-fakeIV	No	0.033

5.2 GNNHAR and GNNHAR-IV

Table 3 reports the validation-tuned neural variants. The key point is that GNNHAR-IV is explicitly estimated and tuned using the validation split: the table includes one-, two-, and three-layer GNNHAR models and their IV counterparts. Among the neural IV models, GNNHAR3L-IV has the lowest test MSE, 0.9591, and QLIKE 0.0007964. GNNHAR2L-IV is also competitive, with MSE 1.0180 and QLIKE 0.0008120. The IV channel is important: the corresponding no-IV neural models have higher losses. The tuned neural results therefore change the interpretation from “untuned GNNHAR-IV is fragile” to “GNNHAR-IV is competitive when hyperparameters are selected on validation data.”

5.3 Model Confidence Set and DM Tests

Table 4 reports the MCS results at the 5% level. The confidence set contains GHAR-IV, HAR-IV, GNNHAR3L-IV, GNNHAR2L-IV, HAR, GHAR, and the main fake-IV and random-graph controls. Within the included set, GHAR-IV has the largest MCS p-value, but GNNHAR3L-IV and HAR-IV are also retained with high p-values. This supports the view that the best linear and tuned neural IV models are statistically close by QLIKE in this sample.

Table 5 reports targeted DM comparisons. HAR versus GHAR is not statistically significant at conventional levels. The comparison between HAR-IV and HAR-fakeIV has $p = 0.0679$, which is suggestive at the 10% level and supports the presence of real IV information. The

Table 5: Diebold-Mariano tests based on QLIKE losses

Comparison	Mean loss A	Mean loss B	DM statistic / p-value
HAR vs GHAR	0.000822	0.000811	0.467 / 0.641
GHAR vs GNNHAR1L	0.000811	0.000848	-1.525 / 0.129
GHAR-IV vs GNNHAR1L-IV	0.000785	0.000835	-2.175 / 0.031
HAR-IV vs HAR-fakeIV	0.000793	0.000821	-1.837 / 0.068
GNNHAR1L-IV vs GNNHAR2L-IV	0.000835	0.000812	1.531 / 0.127

Table 6: IV information decomposition based on QLIKE

Family	Baseline	Real IV	Fake IV	Total	Info	Param.
HAR	0.0008220	0.0007928	0.0008207	0.0000292	0.0000279	0.0000013
GHAR	0.0008111	0.0007849	0.0008136	0.0000262	0.0000287	-0.0000025
GNNHAR1L	0.0008476	0.0008348	0.0008702	0.0000128	0.0000354	-0.0000226

comparisons involving tuned GNNHAR models show a more nuanced picture: GNNHAR1L is not significantly worse than GHAR at conventional levels, while GNNHAR1L-IV has higher QLIKE than GHAR-IV. GNNHAR2L-IV improves on GNNHAR1L-IV, but the difference is not statistically significant at the 5% level.

5.4 Does IV Contain Genuine Information?

The fake-IV decomposition in Table 6 is the central extension beyond Zhang et al. For HAR and GHAR, most of the total IV improvement comes from genuine information gain. For HAR, the total QLIKE improvement is 2.915×10^{-5} , of which 2.789×10^{-5} is attributed to genuine information. For GHAR, the genuine information gain is 2.868×10^{-5} . The parameter expansion term is small for HAR and negative for GHAR, meaning that fake IV does not explain the real-IV gains.

For GNNHAR1L, real IV also improves QLIKE relative to the no-IV neural baseline, and the fake-IV control is worse than real IV. This suggests that the IV channel contains information for the neural model as well, although the strongest neural result comes from the deeper GNNHAR3L-IV specification rather than the one-layer decomposition benchmark.

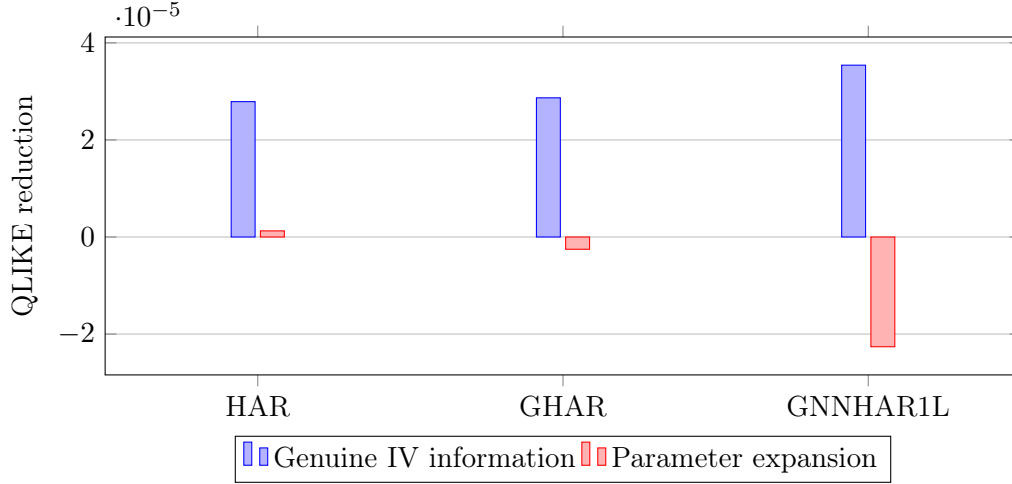


Figure 1: Decomposition of IV gains into genuine information and parameter expansion.

5.5 Regime-Specific Performance

Table 7 reports QLIKE losses in calm and volatile regimes. The volatile regime is defined as the top quartile of market-average realized volatility in the test period. GHAR-IV is the best model in both regimes. Its advantage is economically more visible in the volatile regime, where its QLIKE is 0.001168, compared with 0.001187 for HAR-IV, 0.001211 for GHAR, and 0.001244 for HAR. This regime evidence is consistent with the interpretation that IV is especially useful when option markets price forward-looking risk during high-volatility periods.

5.6 Visual Summary

Figure 2 summarizes QLIKE ratios relative to HAR for the main linear models. Values below one indicate stronger out-of-sample QLIKE performance.

Table 7: Regime-specific QLIKE losses

Model	Calm QLIKE	Volatile QLIKE
GHAR-IV	0.000655	0.001168
HAR-IV	0.000660	0.001187
GHAR	0.000676	0.001211
GHAR-random	0.000678	0.001218
GHAR-fakeIV	0.000678	0.001215
HAR-fakeIV	0.000678	0.001241
HAR	0.000679	0.001244

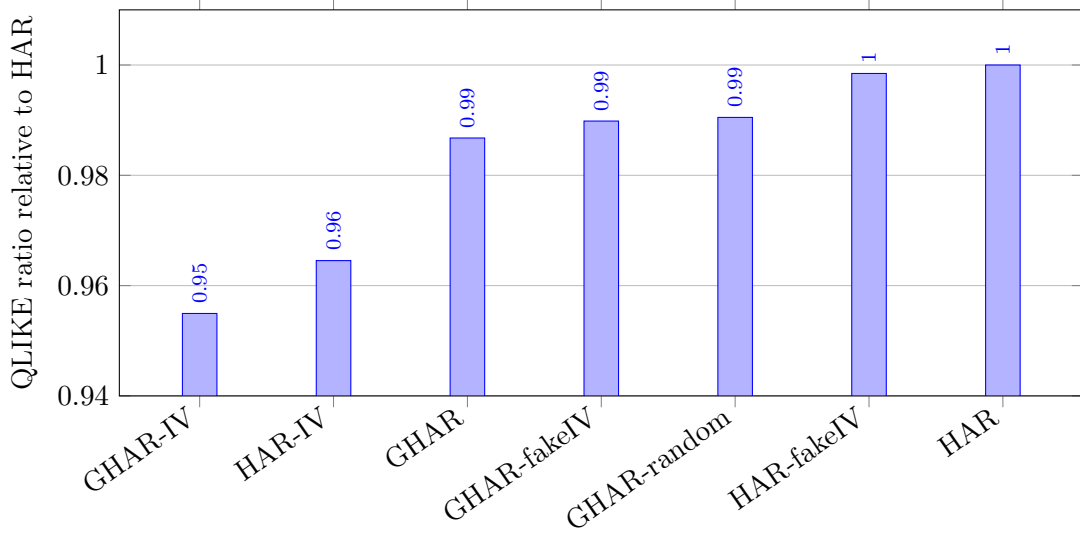


Figure 2: Out-of-sample QLIKE ratios relative to HAR for the main linear model family.

5.7 Distributional Diagnostics Following Zhang et al.

Zhang et al. use grouped boxplots of forecast errors and forecast ratios to diagnose whether models over-predict or under-predict realized volatility across calm and turbulent regimes. We replicate this diagnostic in Figure 3. The left column reports $\hat{v}_{i,t} - v_{i,t}$, while the right column reports $\hat{v}_{i,t}/v_{i,t}$. The dashed vertical lines mark zero forecast error and a unit forecast ratio. In the present Dow 30 experiment, the linear HAR/GHAR family remains tightly centered around these benchmarks, and the validation-tuned GNNHAR-IV variants remain competitive rather than collapsing into the much wider error distributions seen before tuning. This visual evidence is consistent with the loss tables: IV is valuable in the linear graph family and can also help nonlinear graph models when their hyperparameters are selected on the validation window.

Figures 4–6 separate two different visual questions. Figure 4 focuses on loss improvements relative to HAR. It shows that GHAR-IV has the largest average QLIKE reduction, that real-IV gains dominate fake-IV gains in the HAR and GHAR families, and that the cumulative test-period improvement remains positive after the early volatile observations. This is the figure that should be used to compare forecast accuracy across the close linear specifications.

Figure 5 instead asks whether the residual distributions have different shapes. The answer is mostly no: the standardized Q-Q curves for HAR, HAR-IV, GHAR, and GHAR-IV are nearly

Grouped forecast error and ratio boxplots

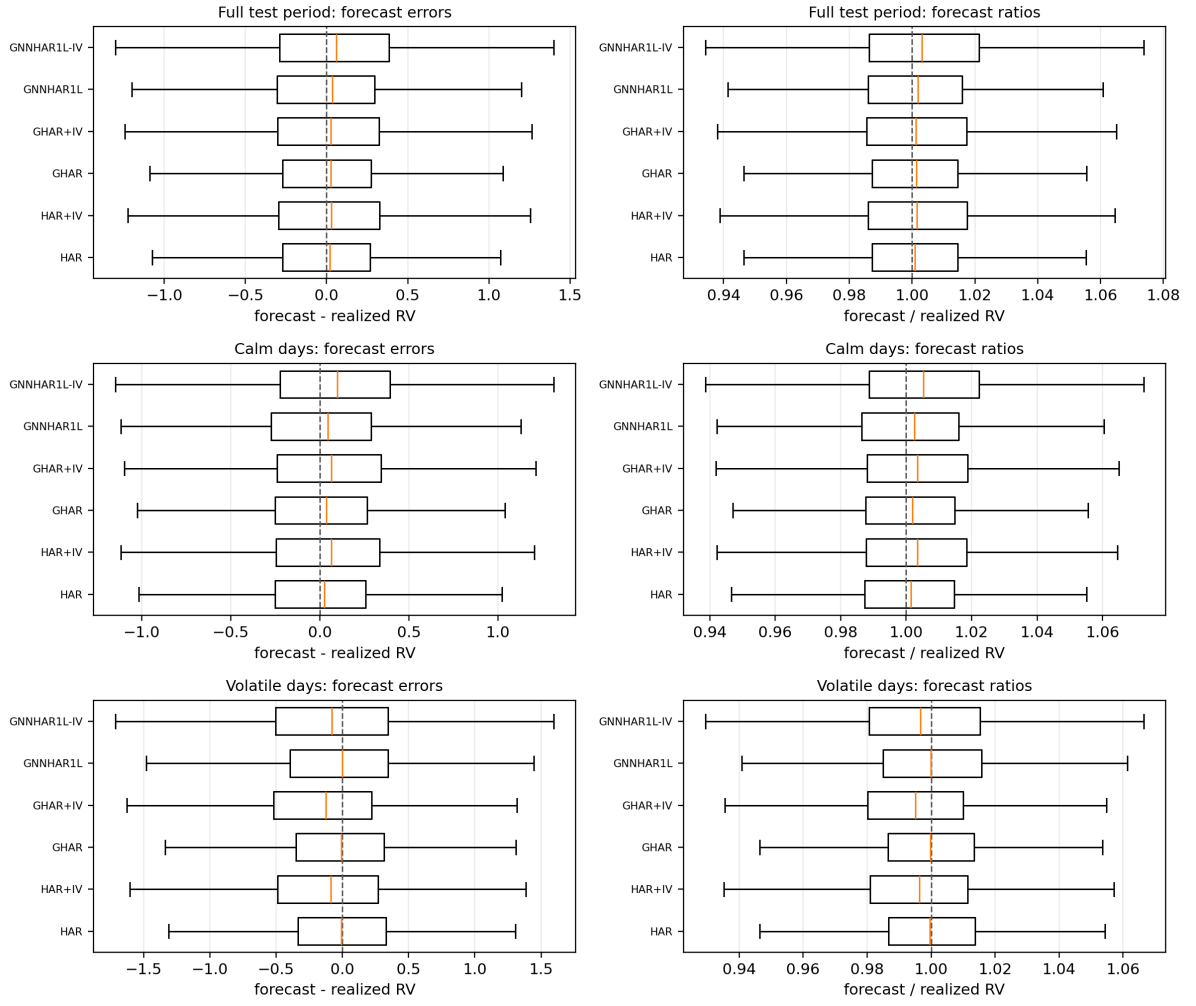


Figure 3: Grouped forecast error and forecast ratio boxplots following the diagnostic layout of Zhang et al. The rows report the full test period, calm days, and volatile days.

overlapping and all exhibit heavy right tails. This is useful diagnostically because it shows that IV and graph spillovers reduce average forecast loss without eliminating the common tail-risk problem in realized-volatility forecasting. The right panels of Figure 5 therefore report quantile differences and tail shifts relative to HAR, rather than presenting four nearly indistinguishable Q-Q panels. Figure 6 compares realized RV with forecast RV; points close to the 45° line correspond to well-calibrated forecasts.

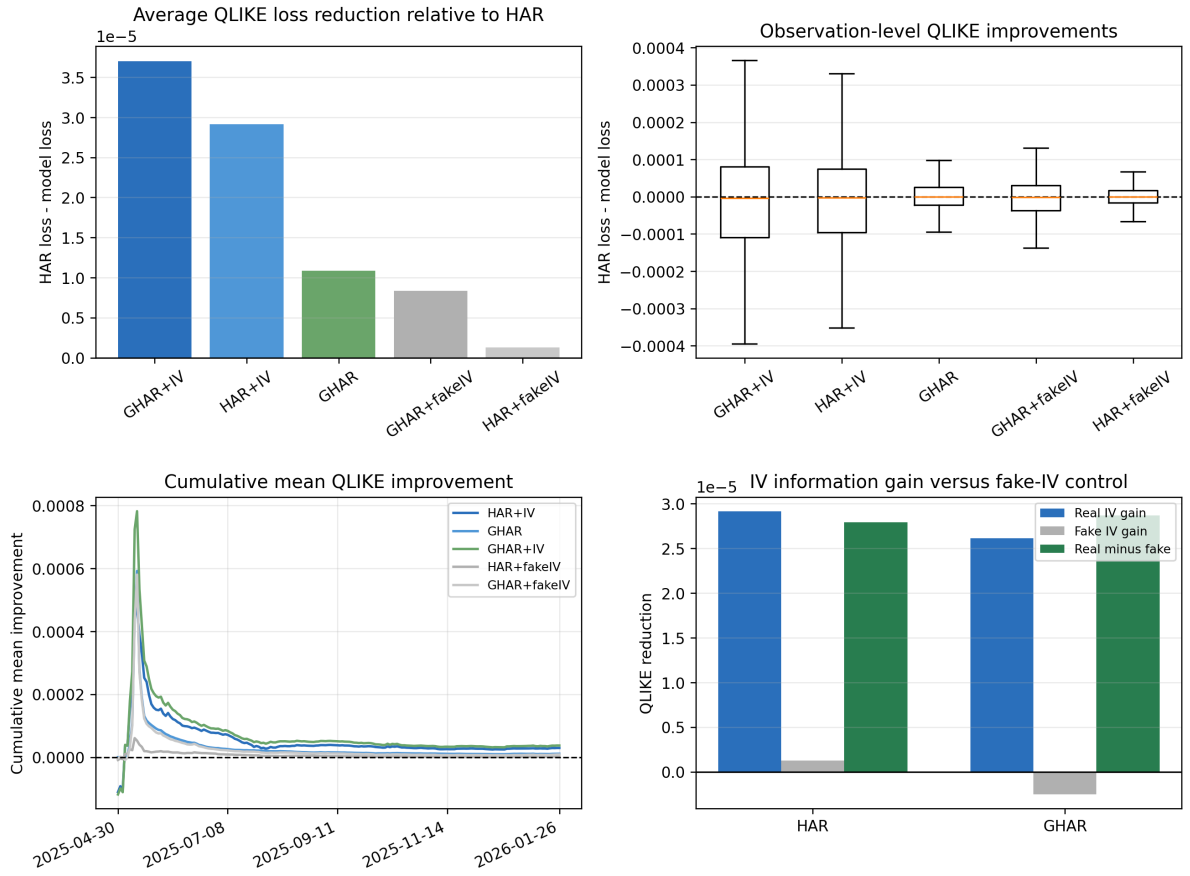


Figure 4: Incremental QLIKE diagnostics relative to HAR. The panels report average QLIKE loss reductions, observation-level improvement distributions, cumulative mean improvements over the test period, and real-IV gains compared with fake-IV controls.

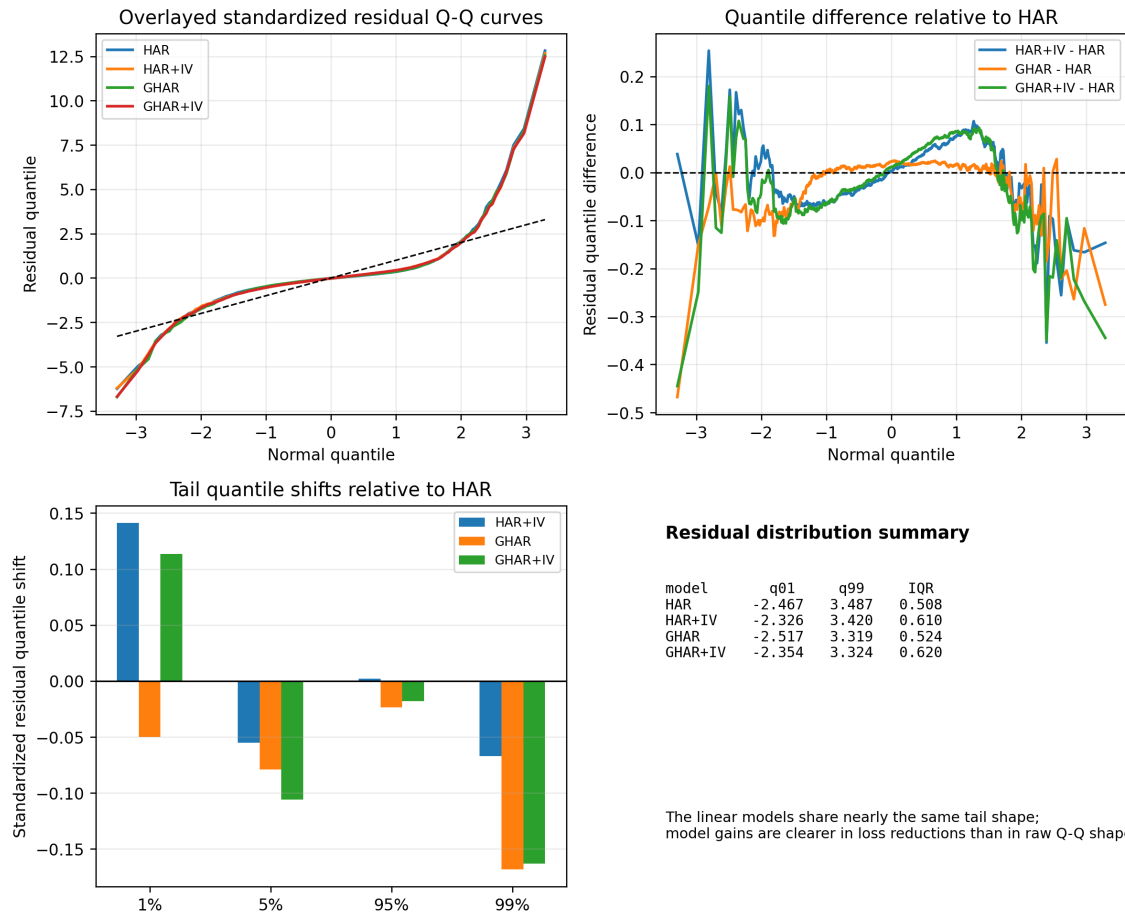


Figure 5: Differential Q-Q diagnostics for standardized forecast residuals. The overlay shows that the four linear specifications have very similar residual tail shapes; the difference and tail-shift panels isolate the small departures relative to HAR.

Forecast-realization phase plots

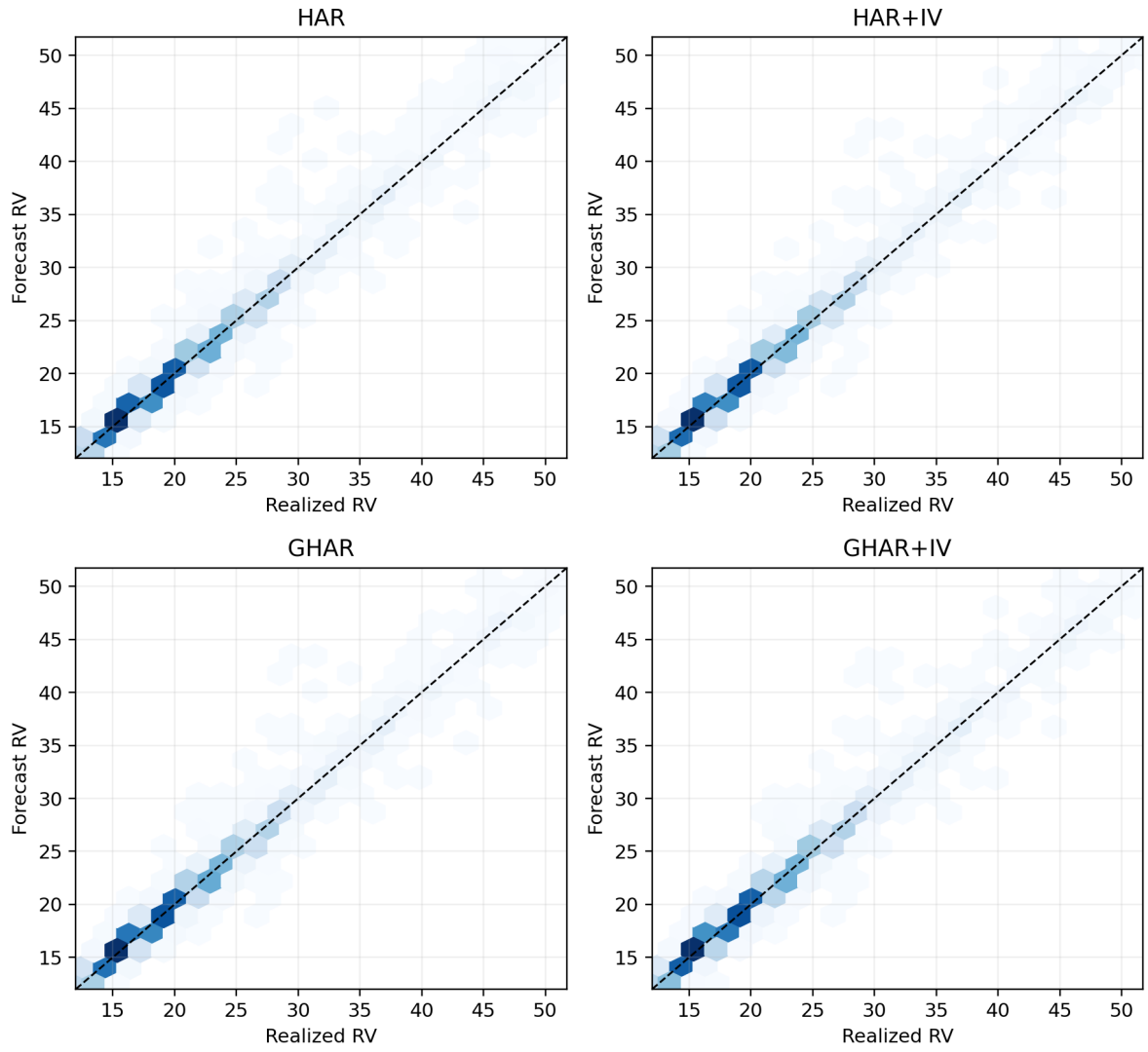


Figure 6: Forecast-realization phase plots for the main linear models.

6 Discussion

The empirical evidence supports three conclusions. First, implied volatility adds predictive content to graph-spillover realized-volatility models. The strongest evidence comes from the HAR and GHAR fake-IV comparisons, where real IV dominates fake IV. This matters because it rules out the simplest explanation that gains arise only because IV models contain more features.

Second, graph spillovers and IV are complementary in the linear family. GHAR improves on HAR, HAR-IV improves on HAR, and GHAR-IV improves on both. This is the cleanest extension of Zhang et al.’s framework: rather than replacing graph spillovers, IV enriches the information set on which spillovers operate.

Third, nonlinear graph neural networks are sensitive to tuning in the Dow 30 setting. Zhang et al. study a larger universe and report that nonlinear GNNHAR models can improve forecasting accuracy. In the present Dow 30 run, validation-tuned GNNHAR3L-IV delivers the lowest MSE, while GHAR-IV remains the best model by QLIKE. This split ranking is useful: neural graph layers help capture large realized-volatility movements, especially in volatile regimes, while the linear GHAR-IV benchmark remains slightly better under the scale-sensitive QLIKE criterion.

7 Robustness and Limitations

Several robustness checks are built into the analysis. First, the random-graph GHAR benchmark performs slightly worse than GLASSO GHAR, indicating that the return-based network contains useful structure. Second, fake-IV controls show that real-IV gains are not explained by feature dimension alone in HAR and GHAR. Third, the regime split shows that GHAR-IV remains strong in calm periods, while GNNHAR3L-IV performs best in the volatile top quartile. Fourth, the GNN tuning table records validation MSE, validation QLIKE, and the selected hidden size, learning rate, and epoch budget for each neural specification.

The study has limitations. The Dow 30 universe is much smaller than the S&P 100 setting in Zhang et al., so the graph neural networks have less cross-sectional information. The IV data are taken from AlphaQuery’s summarized option-statistics fields rather than reconstructed from raw option chains; therefore, vendor definitions, data availability, and interpolation choices may affect results. The neural models use a compact validation grid rather than a very large hyperparameter sweep. Finally, Colab neural results can remain subject to runtime stochasticity even with fixed seeds; the reported tuned tables here are produced by the repository pipeline, and the Colab notebook has been updated to reproduce the same tuning workflow.

8 Conclusion

This paper develops and evaluates a GNNHAR-IV extension of Zhang et al.’s graph-spillover realized-volatility forecasting framework. The extension adds implied volatility at the same HAR horizons as realized volatility and evaluates whether the IV channel contributes genuine information beyond parameter expansion. The empirical results are clear for the linear graph family: GHAR-IV is the best model by test QLIKE, real IV dominates fake IV, and the model performs well across regimes. The nonlinear extension also becomes competitive after validation

tuning: GNNHAR3L-IV achieves the lowest test MSE and the best volatile-regime losses. Overall, IV is a valuable addition to the GNNHAR framework. The strongest current interpretation is not that one model dominates under every loss, but that GHAR-IV provides the most stable QLIKE performance while tuned GNNHAR-IV captures additional nonlinear spillover information under MSE and stress-regime evaluation.

A Colab Reproducibility Appendix

The GNNHAR and GNNHAR-IV results in this paper were run on Google Colab, not only on a local machine. The reproducibility notebook is stored in the GitHub repository and can be opened directly in Colab:

[Open the GNNHAR-IV Colab full-run notebook](#). The repository path is `notebooks/gnnhar_iv_colab_full_run.ipynb`.

The notebook clones the repository branch `2026-06-01` from the private GitHub repository, runs the full GNNHAR-IV pipeline with the repository’s tracked Dow 30 input data, writes outputs to `/content/gnnhar_outputs`, and commits those outputs back to the same private repository under `outputs/colab-runs/<run_id>/`. This avoids both `drive.mount` and external cloud-storage state, leaving GitHub as the single source of versioned code, data, and output artifacts.

- Colab GitHub secret: `GITHUB_TOKEN`.
- Private-repository access: `GITHUB_TOKEN` gives Colab read access to the private GitHub repository containing the code and input data.
- GitHub output access: the same token gives Colab write access to push the generated tables, figures, metadata, and report draft back to the private repository.
- Full run flags: `fast=false`, `epochs=250`, `tune_gnn=true`, `skip_qlike_training=true`, and `mcs_bootstrap=60`.
- Sample and split: seed 20260602, 1234 usable dates, and a 863/185/186 train-validation-test split.
- Neural tables: `tables/model_losses.csv` reports one-, two-, and three-layer GNNHAR models and their IV extensions; `tables/gnn_tuning.csv` records the validation-selected neural hyperparameters.

A.1 Notebook Cells

The core notebook cells are reproduced below so that the empirical run is checkable even without relying on screenshots.

Listing 1: Clone the private GitHub branch and set local output directory.

```
import json
import pathlib
import shutil
import subprocess
```

```

import sys
from datetime import datetime, timezone

from google.colab import userdata

REPO_OWNER = 'easyglider'
REPO_NAME = 'GNNHAR'
BRANCH = '2026-06-01'
REPO_DIR = pathlib.Path('/content/GNNHAR')
OUTPUT_DIR = pathlib.Path('/content/gnnhar_outputs')
RUN_ID = datetime.now(timezone.utc).strftime('%Y%m%dT%H%M%SZ')
REPO_OUTPUT_DIR = REPO_DIR / 'outputs' / 'colab-runs' / RUN_ID

GITHUB_TOKEN = userdata.get('GITHUB_TOKEN')
if not GITHUB_TOKEN:
    raise RuntimeError('Missing Colab Secret: GITHUB_TOKEN')

OUTPUT_DIR.mkdir(parents=True, exist_ok=True)
if REPO_DIR.exists():
    shutil.rmtree(REPO_DIR)

netrc_path = pathlib.Path.home() / '.netrc'
netrc_path.write_text(
    f'machine github.com login x-access-token password {GITHUB_TOKEN}\n'
)
netrc_path.chmod(0o600)
repo_url = f'https://github.com/{REPO_OWNER}/{REPO_NAME}.git'
subprocess.run([
    'git', 'clone', '--depth', '1', '--branch', BRANCH, repo_url, str(REPO_DIR),
], check=True)

```

Listing 2: Install missing Colab dependencies if needed.

```

import subprocess
import sys

mods = [
    'numpy', 'pandas', 'sklearn', 'matplotlib', 'torch', 'scipy',
]
packages = {
    'sklearn': 'scikit-learn',
}
missing = []
for mod in mods:
    try:
        __import__(mod)
    except Exception:
        missing.append(mod)
print('missing:', missing)
if missing:
    subprocess.check_call([
        sys.executable, '-m', 'pip', 'install',
        *[packages.get(m, m) for m in missing],

```

```
])
```

Listing 3: Run the full GNNHAR-IV empirical pipeline.

```
subprocess.run([
    'python', str(REPO_DIR / 'scripts/analysis/gnnhar_iv_pipeline.py'),
    '--data-dir', str(REPO_DIR / 'experiments/dow30/data'),
    '--output-dir', str(OUTPUT_DIR),
    '--epochs', '250',
    '--tune-gnn',
], check=True)
```

Listing 4: List outputs and inspect run metadata.

```
import json
import pathlib

for path in sorted(OUTPUT_DIR.glob('**/*')):
    if path.is_file():
        print(path)

print('\nmetadata:')
print(json.dumps(json.loads((OUTPUT_DIR / 'run_metadata.json').read_text()), indent=2))
```

Listing 5: Inspect the GNNHAR and GNNHAR-IV loss table.

```
import pandas as pd

pd.read_csv(OUTPUT_DIR / 'tables/model_losses.csv').head(20)
```

Listing 6: Commit and push Colab outputs back to private GitHub.

```
import shutil
import subprocess

if REPO_OUTPUT_DIR.exists():
    shutil.rmtree(REPO_OUTPUT_DIR)
REPO_OUTPUT_DIR.parent.mkdir(parents=True, exist_ok=True)
shutil.copytree(OUTPUT_DIR, REPO_OUTPUT_DIR)

relative_output = REPO_OUTPUT_DIR.relative_to(REPO_DIR)
subprocess.run([
    'git', '-C', str(REPO_DIR), 'config', 'user.email',
    'colab-runner@users.noreply.github.com',
], check=True)
subprocess.run([
    'git', '-C', str(REPO_DIR), 'config', 'user.name', 'Colab Runner',
], check=True)
subprocess.run(['git', '-C', str(REPO_DIR), 'add', str(relative_output)], check=True)
subprocess.run([
    'git', '-C', str(REPO_DIR), 'commit', '-m',
    f'Add Colab GNNHAR-IV outputs {RUN_ID}',
], check=True)
```

```

plain_url = f'https://github.com/{REPO_OWNER}/{REPO_NAME}.git'
auth_url = (
    f'https://x-access-token:{GITHUB_TOKEN}@github.com/'
    f'{REPO_OWNER}/{REPO_NAME}.git'
)
subprocess.run([
    'git', '-C', str(REPO_DIR), 'remote', 'set-url', 'origin', auth_url,
], check=True)
try:
    try:
        subprocess.run([
            'git', '-C', str(REPO_DIR), 'push', 'origin', f'HEAD:{BRANCH}',
        ], check=True)
    except subprocess.CalledProcessError:
        subprocess.run([
            'git', '-C', str(REPO_DIR), 'pull', '--rebase', 'origin', BRANCH,
        ], check=True)
        subprocess.run([
            'git', '-C', str(REPO_DIR), 'push', 'origin', f'HEAD:{BRANCH}',
        ], check=True)
finally:
    subprocess.run([
        'git', '-C', str(REPO_DIR), 'remote', 'set-url', 'origin', plain_url,
    ], check=True)

```

References

- AlphaQuery. (2026). Volatility and option statistics data variables. <https://www.alphaquery.com/>. Accessed June 2, 2026.
- Bollerslev, T., Hood, B., Huss, J., and Pedersen, L. H. (2018). Risk everywhere: Modeling and managing volatility. *Review of Financial Studies*, 31(7), 2729–2773.
- Corsi, F. (2009). A simple approximate long-memory model of realized volatility. *Journal of Financial Econometrics*, 7(2), 174–196.
- Diebold, F. X., and Mariano, R. S. (1995). Comparing predictive accuracy. *Journal of Business and Economic Statistics*, 13(3), 253–263.
- Engle, R. F. (2002). New frontiers for ARCH models. *Journal of Applied Econometrics*, 17(5), 425–446.
- Friedman, J., Hastie, T., and Tibshirani, R. (2008). Sparse inverse covariance estimation with the graphical lasso. *Biostatistics*, 9(3), 432–441.
- Hansen, P. R., Lunde, A., and Nason, J. M. (2011). The model confidence set. *Econometrica*, 79(2), 453–497.
- Zhang, C., Pu, X., Cucuringu, M., and Dong, X. (2025). Forecasting realized volatility with spillover effects: Perspectives from graph neural networks. *International Journal of Forecasting*, 41, 377–397.